

C# ve WinForms Temel Kontrollerine Giriş

Görsel Programlama - Hafta 2



Masaüstü Uygulamaları: Windows Forms'a Geçiş

Windows Forms (WinForms), C# kullanılarak Windows masaüstü uygulamalarının kullanıcı arayüzlerini oluşturmak için kullanılan bir teknolojidir. Bu haftanın odak noktası, WinForms uygulamalarının "görsel" kısmına odaklanmaktır. WinForms, hızlı prototipleme ve görsel tasarım becerilerini geliştirmek için idealdir. Kontroller (düğmeler, metin kutuları, etiketler) bu arayüzlerin temel yapı taşlarıdır.

Pedagojik Hedefler: Neyi Öğreneceğiz?

Bu dersin ana hedefi, öğrencilerin bir Windows Forms uygulamasının en temel üç kontrolünü öğrenmesidir: Button, Label ve TextBox. Öğrenciler ayrıca, bu kontrollerin en sık kullanılan özelliklerini (Properties) de öğreneceklerdir. Hedefler:

- Temel kontrolleri tanımak ve işlevlerini anlamak.
- Sürükle-bırak yöntemiyle işlevsel arayüzler tasarlayabilme becerisi kazanmak.
- Programın "görsel" kısmına odaklanarak yaratıcılığı teşvik etmek.

Visual Studio Ortamı ve Arayüz Tasarımı

Kullanıcı arayüzünü tasarlamak için Visual Studio (VS) Entegre Geliştirme Ortamı (IDE) kullanılır. VS'nin sunduğu görsel tasarımcı, arayüzü kod yazmadan oluşturmamızı sağlar. Tasarım sürecinde iki ana araç penceresi kullanılır:

1. Toolbox (Araç Kutusu): Kontrolleri forma sürüklediğimiz yer.
2. Properties Penceresi (Özellikler): Seçili kontrolün görünümünü ve davranışını değiştirdiğimiz yer.



Programın İki Temel Katmanı

: Bir program iki temel katmandan oluşur:

1. Arayüz (UI - User Interface): Ne görüldüğü. (Örneğin, bir düğmenin rengi veya bir etiketin metni). Bu katman genellikle Properties penceresi ile yönetilir.
2. Mantık (Logic): Ne yaptığı. (Örneğin, butona tıklandığında ne olacağı). Bu katman C# kod satırları ile yazılır. Bu hafta, neredeyse hiç kod yazılmaması, bu ayrımı sezgisel olarak öğretmek için bilinçli bir tercihtir.

Sürükle-Bırak ile Arayüz Tasarımı Pratiği

Arayüz tasarımı, kontrolleri Toolbox'tan (Araç Kutusu) forma ekleme, fare ile yeniden boyutlandırma ve konumlandırma pratiği ile yapılır. Bu yöntem, öğrencilerin görsel tasarım becerilerini hızla geliştirmesini sağlar. Kontrolün formu üzerindeki yerleşimi, kullanıcı deneyimi açısından kritiktir.



Hizalama Çizgileri (Snap Lines)

Hizalama Çizgileri (Snap Lines)

- İÇERİK: Visual Studio, kontrollere estetik ve profesyonel bir görünüm kazandırmak için hizalama çizgileri (snap lines) sunar. Bu çizgiler, kontrolleri birbirine göre düzgün, eşit aralıklı ve hizalı bir şekilde yerleştirmeye yardımcı olur. Kontrollerin kenarları veya metin taban çizgileri birbirine yaklaştığında otomatik olarak görünürler.



Temel Kontrol 1: Label (Etiket)

Label, kullanıcıya bilgi göstermek için kullanılan bir etikettir. Genellikle kullanıcı tarafından etkileşime girilmeyen pasif bir kontroldür. Kullanım Alanları: BAŞLIKLAR, diğer giriş alanlarının açıklamaları (örneğin, "Kullanıcı Adı:") veya programın ürettiği sonuçları göstermek için idealdir. Label'ın gösterdiği metin, Text özelliği aracılığıyla ayarlanır.

Label Kullanımı: Metin ve Sonuç Gösterimi

Bir Label nesnesinin temel görevi, kullanıcıya net ve anlaşılır geri bildirim sağlamaktır. Örnek Kullanım: Bir hesap makinesi uygulamasında sonucu göstermek için lblSonuc adında bir Label kullanılır. Label'lar, kullanıcı arayüzünde önemli bir yer tutar; karmaşık veriyi basitleştirerek sunarlar.



Temel Kontrol 2: TextBox (Metin Kutusu)

TextBox, kullanıcıdan metin, sayı veya herhangi bir karakter dizisi gibi verileri almak için kullanılan bir giriş alanıdır. Programın kullanıcıdan veri almasının temel ve en yaygın yoludur. Kullanıcı buraya klavye aracılığıyla veri girer. TextBox içeriği de Text özelliği ile alınır veya ayarlanır.

TextBox Kullanımı: Veri Toplama

TextBox'lar, form doldurma, giriş yapma veya arama yapmak gibi çeşitli veri toplama işlemleri için kullanılır. Örneğin, kullanıcı adı girişi için kullanılan metin kutusu. TextBox'ların doğru isimlendirilmesi (örn. txtKullaniciAdi) önemlidir.



Temel Kontrol 3: Button (Düğme)

Button, kullanıcının bir eylemi başlatması, bir işlemi onaylaması veya bir formu kapatması için tıkladığı kontroldür. Programın mantığını (kod tarafındaki olayları) tetikleyen ana unsurdur. Butonun üzerinde görünen yazı, Text özelliği ile ayarlanır. Kullanıcı, butona tıklayarak bir olay (Event) başlatır.

Button Kullanımı: Eylem Tetikleme

Butonlar, uygulamanın etkileşimli kalbi gibidir. Tıklama olayı (Click Event), C# kodunun çalışmasını sağlar. Örnek Kullanım:

- "Giriş Yap" butonu, kullanıcı bilgilerini kontrol etme eylemini tetikler.
- "İptal" butonu, formu kapatma veya girilen veriyi temizleme eylemini tetikler.



Kontrol İsimlendirme Standardı (Naming Convention)

Kodun okunabilirliğini önemli ölçüde artırmak için, kontrollere standart ön ekler kullanarak isimlendirme yapılmalıdır. Bu kural, bir kontrole kod tarafında atıfta bulunulduğunda, onun tipini (Button mı, TextBox mı?) hızlıca anlamayı sağlar.

Kontrol Tipi	Standart Ön Ek	Örnek İsim
Button	btn	btnGiris
TextBox	txt	txtKullaniciAdi
Label	lbl	lblSonuc

En Önemli Ortak Özellik: Name (Kimlik)

Name, bir kontrolün en önemli özelliğidir. Kontrolün kod tarafında o kontrole atıfta bulunmak için kullanılan benzersiz kimliğidir. Bu özellik, kullanıcıya görünmez; sadece programcı tarafından kullanılır. C# kodu, arayüzdeki bir elemanı Name özelliği üzerinden tanır ve manipüle eder.



Name Özelliği: İsimlendirme Kuralları

Name özelliği, C# değişken isimlendirme kurallarına uygun olmalıdır:

1. Harfle Başlama: İsimler bir harfle başlamalıdır.
2. Boşluksuz: İsim içinde boşluk karakteri bulunmamalıdır.
3. Türkçe Karakter Olmaması: Ç, Ğ, İ, Ö, Ş, Ü gibi Türkçe karakterler kullanılmamalıdır.
4. Standart Ön Ek Kullanımı: btn, txt, lbl gibi ön ekler okunabilirliği artırır.



Text Özelliği (Görünen Yazı)

Text, kontrolün üzerinde kullanıcıya görünen yazıdır.

- Button için: Düğme üzerindeki eylem adı ("Giriş Yap").
- Label için: Açıklama veya BAŞLIK metni ("Kullanıcı Adı:").
- Form için: Pencerenin BAŞLIK çubuğundaki metin ("Kullanıcı Girişi"). Text özelliği C# dilinde bir string (metin dizisi) tipindedir.

Text Özelliğinin Dinamik Yönetimi

Text özelliği, hem tasarım anında (Properties penceresi) hem de çalışma anında (kod ile) değiştirilebilir. Dinamik yönetim örneği: Program bir işlemi tamamladığında, `lblDurum.Text` özelliği "İşlem Başarılı!" olarak güncellenebilir. Bu, kullanıcıya durum değişikliklerini anında bildirmek için önemlidir.

Renk Özellikleri: BackColor ve ForeColor

BackColor: Kontrolün arka plan rengini belirler. ForeColor: Kontrolün üzerindeki yazının (metnin) rengini belirler. Renk seçimi, arayüzün estetiği ve marka kimliği açısından önemlidir. Aynı zamanda, özellikle görme engelli kullanıcılar için yeterli kontrast sağlamak kritiktir (Erişilebilirlik).



Font Özelliği: Okunabilirlik ve Stil

Font, kontrol üzerindeki metnin yazı tipini, boyutunu ve stilini ayarlayan karmaşık bir özelliktir. Font ayarları şunları içerir:

- Yazı Tipi (örn. Arial, Segoe UI)
- Boyut (örn. 12pt)
- Stil (Kalın/Bold, İtalik/Italic, Altı Çizili). Proje örneğinde, Label'ların font özellikleri kalın (Bold) yapılmıştır.

Font Seçimi ve Kullanıcı Deneyimi (UX)

Üniversite düzeyindeki uygulamalarda, font seçimi yalnızca estetik değil, aynı zamanda kullanıcı deneyimi (UX) ile de ilgilidir. Okunabilirlik (Readability): Karmaşık uygulamalarda, standart ve kolay okunan fontlar tercih edilmelidir. Tutarlılık: Formdaki tüm metinlerin (BAŞLIKLAR hariç) aynı veya uyumlu font özelliklerine sahip olması, profesyonel bir görünüm sağlar.



Boyut Özelliği: Size

: Size özelliği, kontrolün piksel cinsinden boyutunu belirler. İki temel bileşeni vardır:

1. Width (Genişlik)

2. Height (Yükseklik) Boyutlandırma, sürükle-bırak yöntemiyle fare kullanılarak yapılabilir veya Properties penceresinde sayısal değerler girilerek hassas ayarlar gerçekleştirilebilir.

Konum Özelliği: Location

Location özelliği, kontrolün form üzerindeki konumunu belirler. Konum, piksel cinsinden koordinatlar (Sol, Üst) olarak ifade edilir.

- Sol (Left / X koordinatı)
- Üst (Top / Y koordinatı) Koordinat sistemi, formun sol üst köşesini (0, 0) kabul eder.

Enabled Özelliği: Etkileşim Yönetimi

Enabled, kontrolün kullanıcı tarafından etkileşime girilip girilemeyeceğini belirleyen bir boole (Boolean) değeridir.

- True: Kontrol etkileşimli ve kullanılabilir.
- False: Kontrol gri görünür ve tıklama veya metin girişi kabul etmez. Önemli: Enabled = false olan bir butona tıklanamaz.

Enabled Kullanım Senaryoları

Enabled özelliğinin dinamik olarak yönetilmesi, iş akışı yönetiminde kritik öneme sahiptir:

- Kullanıcı tüm zorunlu alanları doldurana kadar "Kaydet" butonunu devre dışı bırakmak (`btnKaydet.Enabled = false`).
- Veri yüklenirken tüm arayüz elemanlarını geçici olarak dondurmak. Bu, kullanıcıya uygulamanın durumu hakkında net geri bildirim verir.

Visible Özelliği: Görünürlük Kontrolü

Visible, kontrolün form üzerinde görünür olup olmadığını belirleyen bir boole değeridir.

- True: Kontrol görünür.
- False: Kontrol gizlenir. Önemli: Visible = false olan bir kontrol formda yer kaplamaz ve görünmez.



Visible Kullanım Senaryoları

Visible özelliğinin temel amacı, karmaşık arayüzleri aşamalı olarak göstermektir.

- Hata Mesajları: Bir Label kontrolünü varsayılan olarak gizlemek, hata oluştuğunda görünür yapmak (lblHata.Visible = true).
- Sekmeler/Paneller: Farklı kullanıcı rollerine veya uygulama durumlarına göre farklı arayüz parçalarını göstermek veya gizlemek.

Properties Penceresi: Arayüzün Ayar Merkezi

Properties penceresi, arayüzün "davranışını" ve "görünümünü" kod yazmadan değiştirmenin bir yoludur. Bu pencere sayesinde, tasarımcılar ve geliştiriciler, kontrollerin varsayılan ayarlarını hızla kişiselleştirebilir. Tüm ortak özellikler (Name, Text, Font, Size, Enabled, Visible vb.) bu pencere üzerinden yapılandırılır.



TextBox'a Özel Özellik 1: MaxLength

MaxLength, kullanıcının metin kutusuna girebileceği maksimum karakter sayısını sınırlar. Bu özellik, veri bütünlüğünü sağlamak için kritik öneme sahiptir. Örneğin, bir TC kimlik numarası alanı için bu değer 11 olarak ayarlanabilir. Kullanıcı belirlenen karakter sayısından fazlasını girmeye çalıştığında, sistem otomatik olarak girişi engeller.



MaxLength Pratiği: Veri Bütünlüğünü Sağlama

Sınırlandırma (MaxLength), uygulamanın arka uç veri tabanı yapısıyla uyumlu olmalıdır. Örneğin, eğer bir veritabanı alanı 50 karakter alabiliyorsa, ilgili TextBox'ın MaxLength özelliği 50'den büyük olmamalıdır. Bu özellik, verilerin yanlış biçimde veya aşırı uzunlukta girilmesini önleyerek hata ayıklamayı kolaylaştırır.



TextBox'a Özel Özellik 2: PasswordChar

PasswordChar, bu özelliğe atanan karakterin (genellikle * veya •) metin kutusuna girilen her karakterin yerine gösterilmesini sağlar. Bu özellik, şifre giriş alanları oluşturmak ve girilen veriyi gizlemek için kullanılır. Proje örneğimizde, txtSifre kontrolünün PasswordChar özelliği * olarak ayarlanmıştır.



PasswordChar Pratiği ve Güvenlik

PasswordChar özelliği, yalnızca görsel bir gizleme mekanizmasıdır. C# tarafında okunan Text değeri yine kullanıcının girdiği gerçek şifredir. Gerçek güvenlik, şifrenin sunucuya iletilmeden önce şifrelenmesiyle (hashing) sağlanır. Bu, arayüz katmanının ötesinde, mantık (Logic) katmanının görevidir.



TextBox'a Özel Özellik 3: Multiline

Multiline, true olarak ayarlandığında, metin kutusu birden fazla satır metin kabul edebilir. Bu özellik, Adres, Açıklama veya not defteri benzeri uzun metin girişleri için kullanışlıdır. Multiline açıldığında, TextBox genellikle fare ile yeniden boyutlandırılabilir ve metin kutusunun yüksekliği artırılabilir.



Multiline Kullanımı ve Kaydırma Çubukları

Çok satırlı metin kutuları, içerik kutuya sığmadığında kaydırma çubuklarına (Scrollbars) ihtiyaç duyar. Multiline aktif edildiğinde, genellikle ek bir özellik olan Scrollbars (Kaydırma Çubukları) ayarı ile yatay ve dikey kaydırma çubuklarının görünüp görünmeyeceği de kontrol edilir (Harici Bilgi).



Etkileşimli Kontrol: ComboBox (Harici Kontrol)

Kaynaklar sadece Button, Label ve TextBox'ı ele alsa da, arayüz tasarımında çok sık kullanılan bir diğer temel kontrol ComboBox'tır (Harici Bilgi). ComboBox, bir metin kutusu ve açılır bir liste kombinasyonudur. Kullanıcının listeden bir öğe seçmesini veya bazen kendi değerini girmesini sağlar (Harici Bilgi).



Etkileşimli Kontrol: CheckBox (Harici Kontrol)

CheckBox, kullanıcının bir veya daha fazla seçeneği açıp kapatmasını (seçili/seçili değil - true/false) sağlayan basit bir boole kontrolüdür (Harici Bilgi). CheckBox, genellikle şartları kabul etmek veya birden fazla özellik seçmek için kullanılır (Harici Bilgi).



Kullanıcı Giriş Ekranı Tasarımı: Proje Başlangıcı

Haftalık projemiz, öğrenilen üç temel kontrolü (Button, Label, TextBox) kullanarak basit bir kullanıcı giriş ekranı tasarlamaktır. Bu aşamada amaç, görsel tasarım becerilerini geliştirmektir; herhangi bir C# kodu yazılmayacaktır. Adım 1: Yeni bir Windows Forms projesi oluşturulur.



Proje Adımı 1: Form Özelliklerini Ayarlama

Formun temel görünümünü düzenliyoruz :

- Formun Text özelliği "Kullanıcı Girişi" olarak değiştirilir. (Bu metin, pencerenin BAŞLIK çubuğunda görünecektir).
- Formun boyutu, giriş bileşenlerine yetecek kadar ayarlanır .
- (İpucu: Formun StartPosition özelliği CenterScreen olarak ayarlanarak uygulama açılışta ekranın ortasında başlatılabilir - Harici Bilgi)

Proje Adımı 2: Açıklama Etiketlerini Ekleme (Label)

Kullanıcıdan beklenecek verileri açıklamak için iki adet Label eklenir:

1. Label 1: Text: "Kullanıcı Adı:".
2. Label 2: Text: "Şifre:". Bu Label'ların Font özellikleri kalın (Bold) yapılabilir, bu da görsel hiyerarşiyi artırır. Ayrıca, Label'ların isimleri lblKullaniciAdi ve lblSifre olarak belirlenmelidir.

Proje Adımı 3: Veri Giriş Alanlarını Ekleme (TextBox)

Veri girişi için iki adet TextBox eklenir:

1. Kullanıcı Adı için: Name özelliği txtKullaniciAdi olarak ayarlanır.
2. Şifre için: Name özelliği txtSifre olarak ayarlanır. Önemli Uygulama: txtSifre kontrolünün PasswordChar özelliği * olarak ayarlanır, böylece şifre gizlenir.



Proje Adımı 4: Eylem Düğmelerini Ekleme (Button)

Eylemleri tetikleyecek iki adet Button eklenir:

1. Giriş Düğmesi: Name özelliği btnGiris, Text özelliği "Giriş Yap" olarak ayarlanır.
2. İptal Düğmesi: Name özelliği btnIptal, Text özelliği "İptal" olarak ayarlanır. Bu düğmeler, kullanıcının işlemi başlatmasını veya durdurmasını sağlar.



Proje Adımı 5: Estetik Düzenleme ve Hizalama

Tüm kontroller, profesyonel bir görünüm kazanacak şekilde düzenlenmelidir.

- Label ve TextBox çiftleri yatay olarak hizalanır.
- Button'lar alt kısımda, birbirine eşit aralıklarla yerleştirilir.
- Visual Studio'nun hizalama çizgileri (snap lines) bu aşamada estetik düzenleme için kullanılır.

Tasarımın Gücü: Neden Kod Yazmıyoruz?

: Bu haftanın amacı, arayüz (Ne görüldüğü) ve mantık (Ne yaptığı) ayrımını pekiştirmektir. Programlamanın sadece kod satırlarından ibaret olmadığını, aynı zamanda planlama ve tasarım içerdiğini anlamak, gelecekte karmaşık mimarileri anlamak için temel bir zihinsel yapı taşı oluşturur. Properties penceresi bu görsel tasarımın anahtarıdır.

İpuçları: Etkili Arayüz İsimlendirmesi

Unutmayınız ki, Name özelliği, kod okunabilirliği için hayati öneme sahiptir.

- İsimlerinizin anlamlı ve açıklayıcı olduğundan emin olun (örneğin, btnKaydet yerine button1 kullanmaktan kaçının).
- Tutarlı ön ek kullanımı (txt, lbl, btn), büyük projelerde kontrol tipini anında ayırt etmenizi sağlar.
- İsimlendirmede boşluk ve Türkçe karakter kullanmaktan kaçının.

Özet ve Kavram Haritası

Bu hafta şunları öğrendik:

- Label: Pasif bilgi gösterimi.
- TextBox: Aktif veri alımı (giriş).
- Button: Eylem tetikleyicisi.
- Ortak Özellikler: Name, Text, Font, Enabled, Visible.
- TextBox Özel Özellikler: MaxLength, PasswordChar, Multiline.
- Tasarım, mantıktan önce gelir ve Properties penceresi tasarımın merkezidir.

Sonraki Adım: Mantığı Geliştirmek

Artık estetik ve işlevsel bir arayüze sahibiz. Sonraki aşamada, bu görsel elemanları C# kod satırlarıyla canlandıracağız. Örneğin, btnGiris butonuna tıklandığında (Click Event), txtKullaniciAdi ve txtSifre alanlarındaki verileri okuyup kontrol eden mantığı yazacağız. Bu, programın ikinci katmanı olan Mantık katmanını oluşturur.



Anahtar Kelimeler

Görsel Programlama - Hafta 2

Çalışma Soruları

Görsel Programlama - Hafta 2

1. Bir Windows Forms uygulamasının en temel üç kontrolü hangileridir?

◦ Cevap: Button, Label ve TextBox.

2. Bir kontrolün kod tarafında çağrılmasını sağlayan benzersiz kimliği hangi özelliktir?

◦ Cevap: Name özelliği.

3. Bir TextBox kontrolüne girilen karakterlerin şifre gibi görünmesini (örneğin * ile) sağlayan özellik hangisidir?

◦ Cevap: PasswordChar özelliği.

4. Visual Studio'da kontrolleri birbirine göre düzgün bir şekilde yerleştirmeye yardımcı olan görsel kılavuzların adı nedir?

◦ Cevap: Hizalama çizgileri (snap lines).

5. Bir kontrolün kullanıcı tarafından etkileşime girilip girilemeyeceğini (tıklanabilir olup olmadığını) belirleyen true/false değerli özellik hangisidir?

◦ Cevap: Enabled özelliği.



6. Standart isimlendirme kurallarına göre TextBox ve Label kontrolleri için kullanılması önerilen ön ekler nelerdir?

- Cevap: TextBox için txt, Label için lbl.

7. Kullanıcının bir metin kutusuna girebileceği maksimum karakter sayısını sınırlayan özellik hangisidir?

- Cevap: MaxLength özelliği.

8. Bir programın iki temel katmanı nedir ve bu katmanlar neyi ifade eder?

- Cevap: Arayüz (Ne görüldüğü) ve Mantık (Ne yaptığı).

9. Bir kontrolün arka plan rengini ve yazı rengini belirleyen özellikler sırasıyla hangileridir?

- Cevap: BackColor ve ForeColor.

10. Bir TextBox kontrolünün birden fazla satır metin kabul etmesini sağlayan özellik nedir?

- Cevap: Multiline özelliği.

Deneme Soruları

Görsel Programlama - Hafta 2

1. Kaynak metinde, öğrencilerin ilk hafta hiç kod yazmamasının "bilinçli bir tercih" olduğu belirtiliyor. Bir programın "Arayüz" ve "Mantık" katmanları arasındaki ayrımı öğretmek açısından bu pedagojik yaklaşımın önemini açıklayınız. Bu yöntem, programlamanın sadece kod yazmaktan ibaret olmadığı fikrini nasıl pekiştirir?
2. Name ve Text özellikleri bir kontrolün temel bileşenleridir. Bu iki özelliğin amaçlarını karşılaştırınız. Biri programcıya, diğeri son kullanıcıya hitap ederken aralarındaki temel işlevsel fark nedir? Neden bir kontrolün her iki özelliğe de ihtiyacı vardır?
3. Bir kullanıcı kayıt formunda, "Kullanım Koşullarını Kabul Ediyorum" onay kutusu (CheckBox) işaretlenmeden "Kaydol" butonunun aktif olmaması isteniyor. Enabled özelliğinin bu tür bir iş akışı yönetiminde nasıl dinamik olarak kullanılabilceğini, kullanıcı deneyimini nasıl iyileştireceğini ve olası uygulama senaryosunu açıklayınız.
4. Visible özelliği, bir kontrolü form üzerinde tamamen gizlemek için kullanılır. Bu özelliği, kullanıcıya sadece ihtiyaç duyduğu anda bilgi sunmak için nasıl etkili bir şekilde kullanabilirsiniz? Örneğin, bir formda yapılan bir işlem sonucunda ortaya çıkan bir hata mesajını göstermek için bu özellikten nasıl yararlanılabilir? Bu yaklaşımın arayüz sadeliğine katkısı ne olurdu?
5. Metinlerde, kontrollere btnGiris veya txtKullaniciAdi gibi standart ön ekler kullanılarak isimlendirme yapılmasının "kodun okunabilirliğini önemli ölçüde artırdığı" vurgulanıyor. Büyük ve karmaşık bir projede, bu tür bir isimlendirme standardına uymanın geliştirme ve bakım süreçlerine sağladığı somut faydaları detaylandırınız.



Özet



Süleyman EZDEMİR

suleyman.ezdemir@giresun.edu.tr

